

```
#include <bits/stdc++.h>

#include <omp.h>

using namespace std;

class Graph {
public:
    int vertices;
    vector<vector<int>> adj_matrix;
    vector<bool> visited;

    Graph(int v) {
        this->vertices = v;
        this->adj_matrix.resize(v);
        this->visited.resize(v, false);
    }

    void add_edge(int u, int v) {
        adj_matrix[u].push_back(v);
        adj_matrix[v].push_back(u);
    }

    void reset_visited() { fill(visited.begin(), visited.end(), false); }

    void serial_bfs(int start_vertex) {
        queue<int> q;
        q.push(start_vertex);
        visited[start_vertex] = true;
```

```

while (!q.empty()) {
    int node = q.front();
    q.pop();
    cout << node << " ";

    for (int neighbour : adj_matrix[node]) {
        if (!visited[neighbour]) {
            visited[neighbour] = true;
            q.push(neighbour);
        }
    }
}

```

```

void serial_dfs(int start_vertex) {
    stack<int> s;
    s.push(start_vertex);
    visited[start_vertex] = true;

```

```

while (!s.empty()) {
    int node = s.top();
    s.pop();
    cout << node << " ";

```

```

for (auto it = adj_matrix[node].rbegin(); it != adj_matrix[node].rend();
    it++) {
    int neighbour = *it;
    if (!visited[neighbour]) {

```

```

        visited[neighbour] = true;
        s.push(neighbour);
    }
}
}
}

```

```

void parallel_bfs(int start_vertex) {
    queue<int> q;
    q.push(start_vertex);
    visited[start_vertex] = true;

    while (!q.empty()) {
        int level_size = q.size();
        vector<int> curr_level;

        for (int i = 0; i < level_size; i++) {
            int node = q.front();
            q.pop();
            cout << node << " ";
            curr_level.push_back(node);
        }
    }
}

```

```

#pragma omp parallel for
for (int i = 0; i < curr_level.size(); i++) {
    int node = curr_level[i];
    for (int neighbour : adj_matrix[node]) {
        if (!visited[neighbour]) {

```

```

        // ensure thread-safe access to visited[]
#pragma omp critical
    {
        visited[neighbour] = true;
        q.push(neighbour);
    }
}
}
}
}

void parallel_dfs_util(int vertex) {
    if (visited[vertex])
        return;

    visited[vertex] = true;
    cout << vertex << " ";

#pragma omp parallel for
    for (int i = 0; i < adj_matrix[vertex].size(); i++) {
        int u = adj_matrix[vertex][i];

#pragma omp critical
        {
            if (!visited[u]) {
#pragma omp task
                parallel_dfs_util(u);
            }
        }
    }
}

```

```
    }  
    }  
}  
}
```

```
void parallel_dfs(int start_vertex) {  
    // parallel section to process each neighbor of the start node  
#pragma omp parallel  
    {  
#pragma omp single  
    {  
        // ensures only one thread starts the root DFS  
        parallel_dfs_util(start_vertex); }  
    }  
}  
};
```

```
int main() {  
    Graph g = Graph(6);  
  
    g.add_edge(0, 1);  
    g.add_edge(0, 2);  
    g.add_edge(1, 3);  
    g.add_edge(1, 4);  
    g.add_edge(2, 4);  
    g.add_edge(3, 5);  
    g.add_edge(4, 5);
```

```
cout << "Serial BFS: ";  
  
double start = omp_get_wtime();  
g.serial_bfs(1);  
  
double end = omp_get_wtime();  
  
cout << endl << "Time: " << (end - start) * 1000 << "ms";  
  
g.reset_visited();
```

```
cout << endl << "Serial DFS: ";  
  
start = omp_get_wtime();  
g.serial_dfs(1);  
  
end = omp_get_wtime();  
  
cout << endl << "Time: " << (end - start) * 1000 << "ms";  
  
g.reset_visited();
```

```
cout << endl << "Parallel BFS: ";  
  
start = omp_get_wtime();  
g.parallel_bfs(1);  
  
end = omp_get_wtime();  
  
cout << endl << "Time: " << (end - start) * 1000 << "ms";  
  
g.reset_visited();
```

```
cout << endl << "Parallel DFS: ";  
  
start = omp_get_wtime();  
g.parallel_dfs(1);  
  
end = omp_get_wtime();  
  
cout << endl << "Time: " << (end - start) * 1000 << "ms" << endl;
```

```
return 0;
```

```
}
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <omp.h>
```

```
using namespace std;
```

```
class ParallelSorting {
```

```
public:
```

```
    vector<int> arr;
```

```
    ParallelSorting(vector<int> v) {
```

```
        arr = v;
```

```
    }
```

```
    void printArray() {
```

```
        for (int x : arr) {
```

```
            cout << x << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
// ----- SEQUENTIAL BUBBLE SORT -----
```

```
void serialBubbleSort() {
```

```
    int n = arr.size();
```

```
    for (int i = 0; i < n - 1; i++) {
```

```

    for (int j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            swap(arr[j], arr[j + 1]);
        }
    }
}
}

```

// ----- PARALLEL BUBBLE SORT -----

// Odd-Even Transposition Sort

```

void parallelBubbleSort() {

```

```

    int n = arr.size();

```

```

    for (int phase = 0; phase < n; phase++) {

```

```

        if (phase % 2 == 0) {

```

```

            // Even phase

```

```

#pragma omp parallel for

```

```

        for (int i = 0; i < n - 1; i += 2) {

```

```

            if (arr[i] > arr[i + 1]) {

```

```

                swap(arr[i], arr[i + 1]);

```

```

            }

```

```

        }

```

```

    }

```

```

    else {

```

```

        // Odd phase

```

```

#pragma omp parallel for

```

```

        for (int i = 1; i < n - 1; i += 2) {

```



```

        if (arr[i] > arr[i + 1]) {
            swap(arr[i], arr[i + 1]);
        }
    }
}
}
}
}

```

// ----- MERGE FUNCTION -----

```

void merge(int left, int mid, int right) {

```

```

    vector<int> temp;

```

```

    int i = left;

```

```

    int j = mid + 1;

```

```

    while (i <= mid && j <= right) {

```

```

        if (arr[i] < arr[j]) {

```

```

            temp.push_back(arr[i++]);

```

```

        } else {

```

```

            temp.push_back(arr[j++]);

```

```

        }

```

```

    }

```

```

    while (i <= mid) {

```

```

        temp.push_back(arr[i++]);

```

```

    }

```

```

while (j <= right) {
    temp.push_back(arr[j++]);
}

for (int k = 0; k < temp.size(); k++) {
    arr[left + k] = temp[k];
}
}

// ----- SEQUENTIAL MERGE SORT -----
void serialMergeSort(int left, int right) {

    if (left < right) {

        int mid = (left + right) / 2;

        serialMergeSort(left, mid);
        serialMergeSort(mid + 1, right);

        merge(left, mid, right);
    }
}

// ----- PARALLEL MERGE SORT -----
void parallelMergeSort(int left, int right, int depth = 0) {

    if (left < right) {

```

```

int mid = (left + right) / 2;

if (depth <= 3) {

#pragma omp parallel sections
    {
#pragma omp section
        parallelMergeSort(left, mid, depth + 1);

#pragma omp section
        parallelMergeSort(mid + 1, right, depth + 1);
    }

} else {

    parallelMergeSort(left, mid, depth + 1);
    parallelMergeSort(mid + 1, right, depth + 1);
}

merge(left, mid, right);
}
};

int main() {

    //vector<int> original = {5, 3, 10, 20, 0, -1};

```

```
vector<int> arr(10000);

for(int i = 0; i < 10000; i++) {
    arr[i] = rand() % 10000;
}

// Sequential Bubble Sort
ParallelSorting s1(arr);

double start = omp_get_wtime();
s1.serialBubbleSort();
double end = omp_get_wtime();

cout << "Sequential Bubble Sort: ";
//s1.printArray();
cout << "Execution Time = " << (end - start) << " seconds\n\n";

// Parallel Bubble Sort
ParallelSorting p1(arr);

start = omp_get_wtime();
p1.parallelBubbleSort();
end = omp_get_wtime();

cout << "Parallel Bubble Sort: ";
//p1.printArray();
cout << "Execution Time = " << (end - start) << " seconds\n\n";
```

```

// Sequential Merge Sort
ParallelSorting s2(arr);

start = omp_get_wtime();
s2.serialMergeSort(0, arr.size() - 1);
end = omp_get_wtime();

cout << "Sequential Merge Sort: ";
//s2.printArray();
cout << "Execution Time = " << (end - start) << " seconds\n\n";

// Parallel Merge Sort
ParallelSorting p2(arr);

start = omp_get_wtime();
p2.parallelMergeSort(0, arr.size() - 1);
end = omp_get_wtime();

cout << "Parallel Merge Sort: ";
//p2.printArray();
cout << "Execution Time = " << (end - start) << " seconds\n";

return 0;
}

```

```

#include<iostream>

#include<vector>

```

```

#include<omp.h>

using namespace std;

// take input 100000

// ===== SEQUENTIAL =====

int seq_min(vector<int>& arr)
{
    int min_val = arr[0];
    for(int i = 0; i < arr.size(); i++)
        if(arr[i] < min_val)
            min_val = arr[i];
    return min_val;
}

int seq_max(vector<int>& arr)
{
    int max_val = arr[0];
    for(int i = 0; i < arr.size(); i++)
        if(arr[i] > max_val)
            max_val = arr[i];
    return max_val;
}

long long seq_sum(vector<int>& arr)
{
    long long sum = 0;
    for(int i = 0; i < arr.size(); i++)
        sum += arr[i];
}

```

```
    return sum;
}
```

```
double seq_avg(vector<int>& arr)
{
    return (double)seq_sum(arr) / arr.size();
}
```

```
// ===== PARALLEL (OPTIMIZED) =====
```

```
/*
```

```
void parallel_all(vector<int>& arr, int &min_val, int &max_val, long long &sum)
```

```
{
    min_val = arr[0];
    max_val = arr[0];
    sum = 0;

    #pragma omp parallel for reduction(min:min_val) reduction(max:max_val)
    reduction(+:sum)

    for(int i = 0; i < arr.size(); i++)
    {
        min_val = min(min_val, arr[i]);
        max_val = max(max_val, arr[i]);
        sum += arr[i];
    }
}
```

```
}
```

```
*/
```

```
int minPar(vector<int>& arr) {
```

```
int min_val = arr[0];

#pragma omp parallel for reduction(min:min_val)
for (int i = 0; i < arr.size(); i++)
    if (arr[i] < min_val) min_val = arr[i];

return min_val;
}
```

```
int maxPar(vector<int>& arr) {
    int max_val = arr[0];

    #pragma omp parallel for reduction(max:max_val)
    for (int i = 0; i < arr.size(); i++)
        if (arr[i] > max_val) max_val = arr[i];

    return max_val;
}
```

```
long long sumPar(vector<int>& arr) {
    long long sum = 0;

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < arr.size(); i++)
        sum += arr[i];

    return sum;
}
```



```

double avgPar(vector<int>& arr) {
    return (double)sumPar(arr) / arr.size();
}

// ===== MAIN =====

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> arr(n);

    // Fill with random values
    for(int i = 0; i < n; i++)
        arr[i] = rand() % 1000;

    // Optional: control threads
    //omp_set_num_threads(4);

    // ----- SEQUENTIAL -----
    double start = omp_get_wtime();

    int min_s = seq_min(arr);
    int max_s = seq_max(arr);
    long long sum_s = seq_sum(arr);

```

```

double avg_s = seq_avg(arr);

double end = omp_get_wtime();

cout << "\n--- Sequential Results ---\n";
cout << "Min: " << min_s
    << "\nMax: " << max_s
    << "\nSum: " << sum_s
    << "\nAverage: " << avg_s << endl;
cout << "Sequential Time: " << end - start << endl;

// ----- PARALLEL -----

start = omp_get_wtime();

int min_p = minPar(arr);
int max_p = maxPar(arr);
long long sum_p = sumPar(arr);
double avg_p = avgPar(arr);

end = omp_get_wtime();

cout << "\n--- Parallel Results ---\n";
cout << "Min: " << min_p
    << "\nMax: " << max_p
    << "\nSum: " << sum_p
    << "\nAverage: " << avg_p << endl;
cout << "Parallel Time: " << end - start << endl;

```

```
    return 0;
}
```

```
%%writefile cuda_program.cu
```

```
#include <bits/stdc++.h>
```

```
#include <cuda_runtime.h>
```

```
using namespace std;
```

```
using namespace std::chrono;
```

```
// ----- VECTOR ADDITION -----
```

```
__global__ void vectorAddCUDA(int *a, int *b, int *c, int n)
```

```
{
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
    if (idx < n)
        c[idx] = a[idx] + b[idx];
}
```

```
void vectorAddCPU(int *a, int *b, int *c, int n)
```

```
{
    for (int i = 0; i < n; i++)
        c[i] = a[i] + b[i];
    // for (int i = 0; i < n; i++)
    //     cout << c[i] << " ";
    // cout << endl;
}
```

```
// ----- MATRIX MULTIPLICATION -----
```

```
__global__ void matrixMulCUDA(int *a, int *b, int *c, int N)
```

```
{  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
    int col = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (row < N && col < N)  
    {  
        int sum = 0;  
        for (int k = 0; k < N; k++)  
            sum += a[row * N + k] * b[k * N + col];  
        c[row * N + col] = sum;  
    }  
}
```

```
void matrixMulCPU(int *a, int *b, int *c, int N)
```

```
{  
    for (int i = 0; i < N; i++)  
        for (int j = 0; j < N; j++)  
        {  
            int sum = 0;  
            for (int k = 0; k < N; k++)  
                sum += a[i * N + k] * b[k * N + j];  
            c[i * N + j] = sum;  
        }  
}
```

```
// for (int i = 0; i < N; i++)
```

```
// {
```

```
//   for (int j = 0; j < N; j++)
```

```

// {
//     cout << c[i * N + j] << " ";
// }
// cout << endl;
//}
}

// ----- MAIN FUNCTION -----

int main()
{
    const int vecSize = 1 << 20; // ~1 million
    //const int matrixSize = 256; // 256x256 matrix
    const int matrixSize = 1024;

    // ----- Vector Addition -----

    int *h_a = new int[vecSize];
    int *h_b = new int[vecSize];
    int *h_c_cpu = new int[vecSize];
    int *h_c_gpu = new int[vecSize];

    for (int i = 0; i < vecSize; ++i)
    {
        h_a[i] = rand() % 100;
        h_b[i] = rand() % 100;
    }

    auto start = high_resolution_clock::now();
    vectorAddCPU(h_a, h_b, h_c_cpu, vecSize);

```

```
auto end = high_resolution_clock::now();

cout << "[Vector Addition - CPU] Time: " << duration_cast<milliseconds>(end -
start).count() << " ms\n";
```

```
int *d_a, *d_b, *d_c;

cudaMalloc(&d_a, vecSize * sizeof(int));
cudaMalloc(&d_b, vecSize * sizeof(int));
cudaMalloc(&d_c, vecSize * sizeof(int));
```

```
cudaMemcpy(d_a, h_a, vecSize * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, vecSize * sizeof(int), cudaMemcpyHostToDevice);
```

```
start = high_resolution_clock::now();

vectorAddCUDA<<<(vecSize + 255) / 256, 256>>>(d_a, d_b, d_c, vecSize);

cudaDeviceSynchronize();

end = high_resolution_clock::now();
```

```
cudaMemcpy(h_c_gpu, d_c, vecSize * sizeof(int), cudaMemcpyDeviceToHost);

cout << "[Vector Addition - GPU] Time: " << duration_cast<milliseconds>(end -
start).count() << " ms\n\n";
```

```
// ----- Matrix Multiplication -----
```

```
int *matA = new int[matrixSize * matrixSize];
int *matB = new int[matrixSize * matrixSize];
int *matC_cpu = new int[matrixSize * matrixSize];
int *matC_gpu = new int[matrixSize * matrixSize];
```

```
for (int i = 0; i < matrixSize * matrixSize; ++i)
{
```

```

    matA[i] = rand() % 100;
    matB[i] = rand() % 100;
}

start = high_resolution_clock::now();
matrixMulCPU(matA, matB, matC_cpu, matrixSize);
end = high_resolution_clock::now();

cout << "[Matrix Multiplication - CPU] Time: " << duration_cast<milliseconds>(end -
start).count() << " ms\n";

int *d_matA, *d_matB, *d_matC;
size_t bytes = matrixSize * matrixSize * sizeof(int);
cudaMalloc(&d_matA, bytes);
cudaMalloc(&d_matB, bytes);
cudaMalloc(&d_matC, bytes);

cudaMemcpy(d_matA, matA, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_matB, matB, bytes, cudaMemcpyHostToDevice);

dim3 threadsPerBlock(16, 16);
dim3 blocksPerGrid((matrixSize + 15) / 16, (matrixSize + 15) / 16);

start = high_resolution_clock::now();
matrixMulCUDA<<<blocksPerGrid, threadsPerBlock>>>(d_matA, d_matB, d_matC,
matrixSize);
cudaDeviceSynchronize();
end = high_resolution_clock::now();
cudaMemcpy(matC_gpu, d_matC, bytes, cudaMemcpyDeviceToHost);

```

```
    cout << "[Matrix Multiplication - GPU] Time: " << duration_cast<milliseconds>(end - start).count() << " ms\n";
```

```
    // Cleanup

    delete[] h_a;

    delete[] h_b;

    delete[] h_c_cpu;

    delete[] h_c_gpu;

    delete[] matA;

    delete[] matB;

    delete[] matC_cpu;

    delete[] matC_gpu;

    cudaFree(d_a);

    cudaFree(d_b);

    cudaFree(d_c);

    cudaFree(d_matA);

    cudaFree(d_matB);

    cudaFree(d_matC);

    return 0;
}
```

```
./nvidia-smi
```

```
./nvcc cuda_program.cu -o cuda_program
```

```
!./cuda_program
```